SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5101

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

Total Marks: 50

Code of Conduct:

*I _**Bharath/192324302**_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

1. Secure Password Storage in Web Applications

A web application stores user passwords and needs to protect them against database breaches.

A. Explain why storing plaintext passwords is insecure.

Plaintext passwords are passwords stored in their original, readable form in a database without any protection or transformation.

For example, consider a web application where users register with their usernames and passwords. The database might store the information as follows:

Username	Password
user1	Bharath@123
user2	Welcome@456
user3	Student@789

In this example, anyone who gains access to the database can directly read the users' passwords.

Why plaintext password storage is insecure:

1. Database breaches: If attackers hack the database, they can immediately obtain all stored passwords.
2. Account compromise: Attackers can use stolen passwords to log in to users' accounts.
3. Password reuse attacks: Many users reuse the same password across multiple websites. A stolen password may allow attackers to access other accounts.
4. Insider threats: Employees or administrators who have database access may be able to view users' passwords.
5. Privacy violations: Password theft can expose sensitive personal, financial, or business information.

Therefore, storing passwords in plaintext creates a serious security risk. Passwords should be stored in a form that cannot be easily recovered, even if the database is compromised.

B. Suggest a secure method for storing passwords and justify your choice.

The recommended method is salted password hashing using Argon2id.

Password hashing converts a password into a fixed-length output using a one-way algorithm. Unlike encryption, hashing is not designed to be reversed to recover the original password.

Argon2id is a password-hashing algorithm designed to make password guessing expensive in terms of computing resources and memory.

Working process:

Step 1: User enters a password

Example: MyPassword@123

Step 2: Generate a unique random salt

A different salt is used for each password.

Step 3: Apply Argon2id

Password + salt + configured hashing parameters

Step 4: Store the hash and salt

The original password is not stored.

How password verification works:

1. When the user registers, the application generates a random salt.
2. The application hashes the password using Argon2id and the salt.
3. The database stores the resulting hash, salt, and necessary algorithm parameters.
4. When the user logs in, the application retrieves the stored values and verifies the newly entered password using the same hashing parameters.
5. If verification succeeds, the user is authenticated. Otherwise, access is denied.

The salt does not need to be secret; its purpose is to ensure that identical passwords produce different stored hashes.

Justification for choosing Argon2id:

- It is designed specifically for password storage.
- Its memory-intensive operation makes large-scale password guessing more expensive.
- Unique salts prevent attackers from efficiently using precomputed rainbow tables and make identical passwords produce different hashes.
- It supports configurable memory, time, and parallelism costs.
- It helps protect passwords even if an attacker obtains the password database.

Conclusion: A web application should never store plaintext passwords. Salted Argon2id password hashing is a strong choice because it reduces the risk of password recovery after a database breach. Bcrypt or scrypt are also established alternatives when configured appropriately.

2. Digital Signature for Document Authentication

A company wants to ensure that official documents sent electronically are authentic and unaltered.

A. Explain how digital signatures can be used in this scenario.

A digital signature is a cryptographic mechanism used to verify the authenticity and integrity of an electronic document. It uses public-key cryptography, involving a private key and a corresponding public key.

Suppose a company sends an official contract electronically to another company. The receiver must confirm that the contract genuinely came from the sender and that nobody modified it during transmission.

A digital signature helps accomplish this.

Working process:

1. Sender prepares the document

Example: An official company contract.

2. Calculate the document hash

A hash algorithm such as SHA-256 produces a digest.

3. Sign the hash using the sender's private key

The result is the digital signature.

4. Send the document and signature

The receiver obtains both items.

5. Verify the signature

The receiver uses the sender's trusted public key and checks the document's hash.

Detailed explanation:

1. The sender creates the official document.
2. A cryptographic hash function calculates a digest of the document.
3. The sender's digital-signature algorithm signs that digest using the sender's private key.
4. The sender transmits the document along with the digital signature.
5. The receiver verifies the signature using the sender's public key and the appropriate signature algorithm.

6. The receiver also checks that the signed content matches the received document. If the document has been changed, verification fails.

Algorithms such as RSA-PSS or ECDSA can be used for digital signatures. SHA-256 can be used as the hash algorithm.

B. Justify how this method ensures both integrity and non-repudiation.

Digital signatures provide two important security properties: integrity and non-repudiation.

1. Integrity

Integrity means that the document has not been modified without detection.

- The sender signs a digest associated with the original document.
- The receiver verifies the signature and the document's contents.
- If an attacker changes even a small part of the document, the digest will normally change.
- The modified document will not pass the signature verification check.

For example, if an attacker changes a contract amount from ₹50,000 to ₹90,000, the modified document will fail verification against the original signature.

2. Non-repudiation

Non-repudiation means providing evidence that helps prevent the signer from falsely denying having signed a document.

- The signature is generated using the sender's private key.
- The corresponding public key can be used to verify the signature.
- If the private key is securely controlled and the public key is reliably linked to the sender's identity, the signature provides evidence of the sender's involvement.
- This evidence can support legal or business disputes.

However, non-repudiation is not absolute proof by itself. Private-key theft, shared access, weak identity verification, or disputes about when a signature was created can affect its evidentiary value. Secure key management, digital certificates, and trusted timestamps can strengthen it.

Conclusion: Digital signatures are suitable for contracts, invoices, certificates, and official documents because they help detect unauthorized changes and provide verifiable evidence of the signer's identity.

3. Preventing Replay Attacks in Online Transactions

An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.

A. Explain how replay attacks occur.

A replay attack is a security attack in which an attacker captures a previously valid message or transaction and sends it again to make the system repeat an action.

In an online payment system, an attacker may capture a valid payment request and attempt to resend it.

Example:

Suppose a customer sends a payment request to transfer ₹1,000 to a merchant.

1. The customer sends a payment request to the server.
2. The attacker captures the request while it is being transmitted.
3. The original payment is processed successfully.
4. The attacker sends the same request to the server again.
5. If the server does not detect duplicate or outdated requests, it may process the payment a second time.

This can result in duplicate payments, unauthorized repeated actions, or fraudulent transactions.

Common causes of replay attacks:

- No unique transaction identifier.
- No timestamp or expiration time.
- No mechanism to track previously processed requests.
- Failure to validate message freshness.
- Inadequate transaction authentication.

Encryption alone does not necessarily prevent replay attacks. An attacker may replay a previously valid encrypted message without decrypting it if the receiver accepts it again.

B. Suggest a cryptographic technique to prevent such attacks and justify your answer.

A suitable solution is to use cryptographic nonces, message authentication codes (MACs), and server-side replay detection.

A nonce is a unique value used for a particular request. It helps the receiver determine whether a message is fresh or has already been processed.

A MAC, such as HMAC-SHA-256, authenticates a message using a shared secret key and helps detect unauthorized changes.

Working process:

1. Customer creates a payment request

Amount + recipient + unique transaction ID + nonce.

2. Authenticate the request

Generate a MAC over all important transaction fields.

3. Server validates the request

Check the MAC, nonce, timestamp, and transaction ID.

4. Accept once and record the transaction

Reject duplicate requests or previously used nonces.

Detailed explanation:

1. The customer creates a payment request containing the transaction details and a fresh nonce or unique transaction ID.
2. The request is authenticated using a MAC generated with a secret key shared between the authorized parties, or by another suitable authenticated protocol.
3. The server verifies the MAC to ensure that the authenticated fields have not been changed.
4. The server checks whether the nonce or transaction ID has already been used.
5. If the request is new and valid, the server processes it and records the identifier.
6. If the same request is received again, the server rejects it or returns the original transaction result without processing another payment.

The transaction record and payment operation should be handled atomically to prevent two simultaneous duplicate requests from being processed.

Justification:

- Nonce: Helps ensure that each request is fresh.
- MAC: Detects changes to authenticated transaction details and verifies message authenticity.
- Timestamp and expiry: Limit how long a request remains valid.
- Transaction ID: Helps the server identify duplicate payment requests.
- Replay tracking: Prevents previously accepted requests from being executed again.

A nonce alone is not enough if the server does not check and record its use. Similarly, a timestamp alone is insufficient because the same request could be replayed within the allowed time window.

Conclusion: A nonce-based authenticated transaction protocol with server-side duplicate detection is an effective defense against replay attacks. HTTPS should also be used to protect payment requests during transmission.

4. Secure Communication over Public Wi-Fi

A user accesses sensitive information using a public Wi-Fi network without encryption.

A. Identify the risks involved in this scenario.

Public Wi-Fi is commonly available in airports, railway stations, hotels, shopping malls, and cafés. It is convenient, but it can expose users to security threats, especially when sensitive information is transmitted without encryption.

Major risks:

1. Eavesdropping

An attacker connected to the same network may be able to monitor unencrypted traffic.

For example, if a website sends login credentials in plaintext, an attacker who can observe that traffic may obtain the username and password.

2. Man-in-the-middle attacks

An attacker may position themselves between the user and the website or service. They may attempt to intercept communications, alter messages, or impersonate one side of the connection.

3. Fake Wi-Fi hotspots

Attackers may create a network with a name similar to a legitimate hotspot. Users who connect to the fake network may expose their traffic to the attacker.

4. Session hijacking

If session tokens or cookies are exposed, an attacker may use them to impersonate an authenticated user, depending on how the application protects those tokens.

5. Data theft and privacy loss

Unprotected traffic may reveal personal information, login credentials, messages, or sensitive business data.

6. Malicious redirection

An attacker may attempt to redirect a user to a fraudulent website or manipulate unprotected traffic to display malicious content.

Conclusion: Using public Wi-Fi without appropriate encryption increases the possibility of data interception, identity theft, account compromise, and unauthorized access.

B. Recommend a secure communication method and justify how it protects user data.

The primary recommendation is HTTPS using TLS (Transport Layer Security). A reputable VPN can provide an additional layer of protection for network traffic, particularly where application-level encryption is absent, but it does not replace HTTPS.

1. HTTPS with TLS

HTTPS is the secure version of HTTP. TLS establishes an authenticated, encrypted connection between the user's device and the website.

Working process:

User's device

Sends a request to the website.

TLS secure connection

Authenticates the server, negotiates keys, and protects data in transit.

Website server

Receives and responds over the protected connection.

How HTTPS protects data:

- Confidentiality: TLS encrypts data in transit so that people monitoring the network cannot normally read the protected content.
- Integrity: TLS detects unauthorized modification of protected messages.

- Authentication: Certificates help the browser verify the identity of the website it connects to.
- Protection against interception: Correctly configured HTTPS makes it much harder for an attacker to read or modify communication without detection.

Users should check that the browser displays HTTPS and a valid certificate, avoid ignoring certificate warnings, and ensure that the domain name is correct.

2. VPN as an additional measure

A Virtual Private Network (VPN) creates an encrypted tunnel between the device and a VPN server.

A VPN can help protect traffic from local network observers, including on public Wi-Fi. However, the VPN provider may be able to observe some traffic metadata, and a VPN does not guarantee protection against phishing, malware, or compromised websites.

Additional safety measures:

1. Disable automatic connection to unknown Wi-Fi networks.
2. Avoid sensitive transactions on suspicious hotspots.
3. Keep the operating system, browser, and applications updated.
4. Enable multi-factor authentication for important accounts.
5. Use HTTPS even when connected to a VPN.
6. Avoid installing certificates or profiles supplied by unknown network operators.

Conclusion: HTTPS with TLS is the essential method for securing communication with websites over public Wi-Fi. A trusted VPN can add protection against local network monitoring, while safe browsing practices further reduce risk.

5. Data Integrity in Cloud Storage

An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

A. Explain how hashing can be used to verify data integrity.

Hashing is a cryptographic process that converts data of any size into a fixed-length value called a hash or message digest.

An organization can use a cryptographic hash algorithm such as SHA-256 to check whether a file stored in the cloud has changed.

A small modification to the file will normally produce a different hash value.

Example:

Suppose an organization uploads an important financial report to cloud storage.

- Original file: financial_report.pdf
- Hash calculated before upload: H1
- Hash calculated after download: H2

The organization compares the two values.

Condition	Result
$H1 = H2$	No change is detected in the file contents, assuming a secure hash algorithm.
$H1 \neq H2$	The file contents have changed or the wrong file was checked.

Working process:

1. Calculate the original file hash

Apply SHA-256 to the file before uploading.

2. Store the file in the cloud

Keep the original hash in a trusted location.

3. Download or retrieve the file

Calculate its current SHA-256 hash.

4. Compare the hashes

Matching hashes indicate that no change was detected.

Important properties of cryptographic hashing:

1. Fixed-length output: SHA-256 produces a 256-bit hash regardless of file size.
2. Avalanche effect: A small change to the input normally changes many bits of the output.
3. One-way operation: It should be computationally infeasible to reconstruct the original file from its hash.
4. Collision resistance: It should be computationally infeasible to find two different inputs with the same hash.

Limitation: Hashing alone does not prove that a file is trustworthy. If an attacker can modify both the file and the stored hash, they may replace both with matching values. Therefore, the original hash must be protected or authenticated.

B. Suggest an additional mechanism to enhance trust and justify your answer.

An effective additional mechanism is a keyed Hash-based Message Authentication Code (HMAC), such as HMAC-SHA-256.

HMAC uses a cryptographic hash function together with a secret key shared by authorized systems. Unlike a plain hash, it can help verify both data integrity and authenticity for parties that securely possess the key.

Working process:

1. Organization prepares the file

The original data is ready for storage.

2. Generate an HMAC

Use the file and a securely held secret key.

3. Store the file and authentication tag

Protect the secret key separately from cloud data.

4. Recalculate and verify the HMAC

Reject the file if its tag does not match.

How HMAC enhances trust:

- Integrity: Detects changes to the file because the computed authentication tag will not match.
- Authenticity: Helps verify that the data was authenticated by someone who possesses the secret key.
- Protection against hash replacement: An attacker who can modify cloud data but does not know the secret key cannot normally create a valid replacement HMAC.
- Secure verification: The organization can check files after downloading them or before processing them.

The secret key must be stored securely, such as in a key-management service, and access must be restricted. Anyone who obtains the secret key may be able to generate valid HMACs.

For a system that requires public verification, such as an audit in which external parties should verify records without sharing a secret key, digital signatures may be more appropriate. For long-term cloud auditing, versioning, access controls, immutable backups, and independent audit logs can provide additional protection.

Conclusion: SHA-256 hashing helps detect accidental or unauthorized file changes when the reference hash is trustworthy. HMAC-SHA-256 strengthens the process by providing keyed

authentication, making it more difficult for an attacker without the secret key to alter a file and create a valid authentication tag.

Overall comparison of the five security scenarios

Scenario	**Main security problem**	**Recommended method**	**Security benefit**
Password storage	Password theft	Salted Argon2id	Makes password guessing expensive
Document authentication	Forgery and document modification	Digital signatures	Integrity, signer authentication, and evidence supporting non-repudiation
Online payments	Repeated transaction requests	Nonces, MACs, and replay detection	Prevents duplicate execution of captured requests
Public Wi-Fi	Traffic interception	HTTPS with TLS	Confidentiality, integrity, and server authentication
Cloud storage	Unauthorized file modification	SHA-256 plus HMAC	Change detection and keyed data authentication

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand